

Projet Vision par Ordinateur

Reconnaissance faciale & Chiffres manuscrits

Analyse mathématique approfondie

Groupe de 6 — ENSEM, Licence 3 / Début Master

Ce document contient, pour chaque module mathématique :

- (1) les formules détaillées terme à terme
- (2) une intuition visuelle du mécanisme
- (3) le lien direct avec le code Python
- (4) une fiche résumé par membre du groupe

Contents

1	Répartition du groupe et feuille de route	2
2	Analyse Détaillée de la Détection : MTCNN	5
2.1	Prétraitement : La Pyramide d'Images	5
2.2	Étape 1 : Proposal Network (P-Net)	6
2.3	Étape 2 : Refine Network (R-Net) — Calibration de boîte	6
2.4	Étape 3 : O-Net — Régression des Landmarks	8
3	Extraction et Signature : FaceNet	9
3.1	La fonction d'embedding et l'hypersphère	9
3.2	Triplet Loss	11
4	Classification SVM Multi-Classes	13
4.1	Problème d'optimisation Soft Margin	13
4.2	Stratégie One-vs-One (OvO)	16
5	Chiffres Manuscrits : Extraction de Features (P5)	17
5.1	De l'image au vecteur	17
5.2	Réduction de dimension par PCA	17
6	Évaluation & Métriques (P6)	19
6.1	Matrice de confusion et métriques dérivées	19

1 Répartition du groupe et feuille de route

Cette section présente la répartition mathématique du groupe. Chaque personne possède un *bloc autonome* avec des interfaces claires vers les autres membres.

Personne 1 — Prétraitement & détection (P-Net)

Projet : Reconnaissance faciale

Maths à maîtriser :

- Produit de convolution discret 2D et notion de noyau (12×12)
- Construction de la pyramide d'images (facteur d'échelle s)
- Score de confiance et seuillage (*Non-Maximum Suppression*)
- Régression de boîte : décalages relatifs $(\delta x, \delta y, \delta w, \delta h)$

Ce que tu fournis à P2 : la liste des fenêtres candidates (x, y, w, h, p)

Personne 2 — Raffinement & landmarks (R-Net / O-Net)

Projet : Reconnaissance faciale

Maths à maîtriser :

- Calibration de boîte : correction exponentielle (e^{t_w}, e^{t_h})
- Fonction de perte landmarks $\mathcal{L}_{\text{land}} = \sum_{i=1}^5 \|\hat{L}_i - L_i\|_2^2$
- Perte multi-tâche pondérée : $\mathcal{L} = \alpha \mathcal{L}_{\text{face}} + \beta \mathcal{L}_{\text{box}} + \gamma \mathcal{L}_{\text{land}}$
- Norme euclidienne et différentiabilité ℓ_2^2 vs ℓ_1

Tu reçois de P1 : candidats bruts. **Tu envoies à P3 :** image de visage recadrée 160×160

Personne 3 — Embedding & métrique (FaceNet)

Projet : Reconnaissance faciale

Maths à maîtriser :

- Fonction d'encodage $f : \mathbb{R}^{160 \times 160 \times 3} \rightarrow \mathbb{R}^{512}$

- Normalisation L2 et géométrie sphérique : $\|f(x)\|_2 = 1$
- Triplet Loss avec ancre / positif / négatif et marge α
- Distance euclidienne au carré et similarité cosinus

Tu reçois de P2 : image 160×160 . **Tu envoies à P4 :** vecteur $\in \mathbb{R}^{512}$

Personne 4 — Classification SVM (commun aux 2 projets)

Projets : Reconnaissance faciale *et* Chiffres manuscrits

Maths à maîtriser :

- Problème d'optimisation Soft Margin : $\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C \sum \xi_i$
- Notion d'hyperplan, marge, variables d'écart ξ_i
- Stratégie One-vs-One et règle de vote $\hat{y} = \arg \max_c \sum_{j < k} \mathbb{1}\{f_{j,k} = c\}$
- Noyau RBF (pour extension non-linéaire)

Tu reçois de P3 : vecteur 512D (visages) et de P5 : vecteur compact (chiffres)

Personne 5 — Features chiffres manuscrits (MNIST)

Projet : Chiffres manuscrits

Maths à maîtriser :

- Image MNIST = vecteur de $28 \times 28 = 784$ pixels $\in [0, 1]^{784}$
- Analyse en Composantes Principales (PCA) : diagonalisation de la matrice de covariance
- Valeurs propres / vecteurs propres et sélection des k premières composantes
- Gradient d'image et descripteur HOG (alternative à PCA)

Tu envoies à P4 : vecteur réduit (ex. 50D au lieu de 784D)

Personne 6 — Évaluation & métriques (commun aux 2 projets)

Projets : Reconnaissance faciale *et* Chiffres manuscrits

Maths à maîtriser :

- Matrice de confusion C_{ij} : vrais positifs, faux positifs, etc.
- Précision = $\frac{VP}{VP+FP}$, Rappel = $\frac{VP}{VP+FN}$, $F_1 = 2 \cdot \frac{P \cdot R}{P+R}$
- Courbe ROC et AUC (aire sous la courbe)
- Validation croisée k -fold et estimation de la variance de généralisation

Tu évalues : les sorties de P4 pour les deux projets

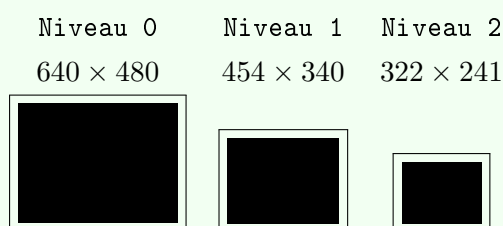
2 Analyse Détaillée de la Détection : MTCNN

2.1 Prétraitement : La Pyramide d'Images

Avant toute détection, l'image est redimensionnée de façon itérative pour former une **pyramide d'images**.

Intuition visuelle — la pyramide d'échelles

Imaginons une image 640×480 . On applique un facteur d'échelle $s = 0.709$ à chaque étape :



Un visage de 100×100 pixels sur le niveau 0 devient $\approx 71 \times 71$ au niveau 1, et $\approx 50 \times 50$ au niveau 2. Le réseau P-Net travaille toujours avec des noyaux 12×12 : c'est la pyramide qui adapte la taille des visages, pas le réseau.

Lien avec le code Python

Listing 1: Construction de la pyramide d'images (P1)

```
1 import numpy as np
2 from PIL import Image
3
4 def build_image_pyramid(img, min_face=20, factor=0.709):
5     """
6     Construit la pyramide d'échelles.
7     - img      : image PIL originale
8     - min_face : taille minimale d'un visage à détecter (pixels)
9     - factor   : facteur de réduction entre deux niveaux (0 <
10                  factor < 1)
11     Retourne une liste d'images redimensionnées.
12     """
13     h, w = img.size[1], img.size[0]
14
15     # Echelle de départ : le plus grand facteur tel que
16     # la plus petite dimension reste >= 12 (taille du noyau P-Net)
```

```

16     scale = 12.0 / min_face          # ex: 12/20 = 0.6
17     pyramid = []
18
19     while min(h, w) * scale >= 12:    # on s'arrete quand l'image
        est trop petite
20         new_w = int(w * scale)
21         new_h = int(h * scale)
22         # Redimensionnement bilineaire
23         resized = img.resize((new_w, new_h), Image.BILINEAR)
24         pyramid.append((resized, scale))
25         scale *= factor                # echelle suivante : encore
        plus petite
26
27     return pyramid
28     # Resultat : liste de paires (image_reduite, facteur_echelle
        )

```

2.2 Étape 1 : Proposal Network (P-Net)

Le P-Net scanne chaque niveau de la pyramide avec un noyau 12×12 .

Intuition — la convolution comme loupe glissante

Pensez au P-Net comme à une petite loupe de 12×12 pixels qui se déplace sur toute l'image, case par case. À chaque position, elle répond à deux questions :

1. *Y a-t-il un visage ici ?* $\longrightarrow$ score $p \in [0, 1]$
2. *Si oui, comment corriger la boîte ?* $\longrightarrow (\delta x, \delta y, \delta w, \delta h)$

Une image de 100×100 produit environ $89 \times 89 = 7921$ positions à examiner simultanément (grâce à la convolution).

2.3 Étape 2 : Refine Network (R-Net) — Calibration de boîte

Pour chaque fenêtre candidate de coordonnées (x, y, w, h) , le R-Net prédit des corrections relatives :

$$x' = x + t_x \cdot w, \quad y' = y + t_y \cdot h, \quad w' = w \cdot e^{t_w}, \quad h' = h \cdot e^{t_h} \quad (1)$$

Symbole	Signification
(x, y)	Coin supérieur gauche de la boîte originale (en pixels)
(w, h)	Largeur et hauteur de la boîte originale
(t_x, t_y)	Décalage <i>relatif</i> prédit : $t_x = 0.1$ déplace la boîte de $0.1 \times w$ vers la droite
(t_w, t_h)	Log-correction de taille : e^{t_w} est toujours > 0 , ce qui empêche $w' < 0$
(x', y', w', h')	Boîte calibrée, mieux centrée sur le visage

Pourquoi e^{t_w} et pas juste $w + t_w$?

Si on écrivait $w' = w + t_w$, rien n'empêche w' de devenir négatif (réseau non contraint). En passant par l'exponentielle, $w' = w \cdot e^{t_w} > 0$ toujours. C'est une astuce standard en détection d'objets (aussi utilisée dans YOLO, Faster R-CNN).

Lien avec le code Python

Listing 2: Calibration de boîte englobante (P2)

```

1 import numpy as np
2
3 def calibrate_boxes(boxes, offsets):
4     """
5     Applique la calibration de boîte englobante.
6     - boxes : tableau (N,4) avec colonnes [x, y, w, h]
7     - offsets : tableau (N,4) avec colonnes [tx, ty, tw, th]
8         produits par le R-Net
9     Retourne les boîtes calibrées (N,4).
10    """
11    x, y, w, h = boxes[:,0], boxes[:,1], boxes[:,2], boxes[:,3]
12    tx, ty, tw, th = offsets[:,0], offsets[:,1], offsets[:,2],
        offsets[:,3]
13
14    # Eq.(1) : translation relative
15    x_new = x + tx * w          # si tx=0.1 et w=50, on déplace de 5
        pixels
16    y_new = y + ty * h
17
```



```

18     # Eq.(1) : correction exponentielle de taille
19     w_new = w * np.exp(tw) # toujours positif
20     h_new = h * np.exp(th)
21
22     return np.stack([x_new, y_new, w_new, h_new], axis=1)
23
24 # Exemple numerique :
25 box      = np.array([[100, 80, 50, 60]]) # x=100, y=80, w=50, h
      =60
26 offset = np.array([[ 0.1, -0.05, 0.0, 0.1]])
27 print(calibrate_boxes(box, offset))
28 # x_new = 100 + 0.1*50 = 105
29 # y_new = 80 - 0.05*60 = 77
30 # w_new = 50 * e^0.0 = 50
31 # h_new = 60 * e^0.1 = 66.3

```

2.4 Étape 3 : O-Net — Régression des Landmarks

$$\mathcal{L}_{\text{land}} = \sum_{i=1}^5 \left\| \hat{L}_i - L_i \right\|_2^2 \quad (2)$$

Symbole	Signification
$i \in \{1 \dots 5\}$	Les 5 landmarks : œil gauche, œil droit, nez, bouche gauche, bouche droite
$\hat{L}_i = (\hat{x}_i, \hat{y}_i)$	Position prédite par le réseau pour le landmark i
$L_i = (x_i, y_i)$	Position réelle annotée dans les données d'entraînement
$\ \hat{L}_i - L_i\ _2^2$	$(\hat{x}_i - x_i)^2 + (\hat{y}_i - y_i)^2$: distance au carré
$\sum_{i=1}^5$	On cumule l'erreur sur les 5 points

Intuition — erreur de placement d'un point

Imaginons que le landmark « œil gauche » soit réellement en (60, 45) et que le réseau prédit (62, 43). L'erreur partielle est :

$$\|\hat{L}_1 - L_1\|_2^2 = (62 - 60)^2 + (43 - 45)^2 = 4 + 4 = 8$$

Si le réseau était à (70, 45) (erreur plus grande), l'erreur serait $(70 - 60)^2 = 100$ pour la seule composante x . La mise **au carré** punit 12,5× plus cette grosse erreur que la petite. Cela pousse le réseau à éviter les erreurs grossières.

Lien avec le code Python

Listing 3: Calcul de la landmark loss (P2)

```
1 import numpy as np
2
3 def landmark_loss(L_pred, L_true):
4     """
5     Calcule  $L_{land} = \sum_{i=1}^5 ||L_{pred_i} - L_{true_i}||^2_2$ 
6     - L_pred : tableau (N, 5, 2) -- N images, 5 points, coords (
7       x, y)
8     - L_true : tableau (N, 5, 2) -- verite terrain
9     Retourne la loss scalaire moyenne sur le batch.
10    """
11    # Erreur vectorielle pour chaque point
12    diff = L_pred - L_true          # forme (N, 5, 2)
13
14    # Norme euclidienne au carre :  $(dx)^2 + (dy)^2$ 
15    sq_dist = (diff ** 2).sum(axis=2) # forme (N, 5)
16
17    # Somme sur les 5 landmarks
18    per_image_loss = sq_dist.sum(axis=1) # forme (N,)
19
20    # Moyenne sur le batch (convention standard en deep learning)
21    return per_image_loss.mean()
22
23 # Exemple numerique :
24 L_pred = np.array([[62, 43], [100, 44], [80, 70], [65, 95], [95, 95]])
25 # 1 image
26 L_true = np.array([[60, 45], [102, 44], [80, 68], [63, 96], [97, 94]])
27 print(f"Loss = {landmark_loss(L_pred, L_true):.2f}")
28 #  $diff^2$  par point : [8, 4, 4, 5, 5] -> somme = 26
```

3 Extraction et Signature : FaceNet

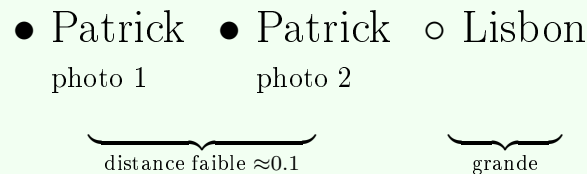
3.1 La fonction d'embedding et l'hypersphère

$$f : \mathbb{R}^{160 \times 160 \times 3} \longrightarrow \mathbb{R}^{512}, \quad \|f(x)\|_2 = 1 \quad (3)$$

Symbole	Signification
$\mathbb{R}^{160 \times 160 \times 3}$	Espace d'entrée : image couleur de 160×160 pixels (76 800 valeurs)
$\mathbb{R}^{512}$	Espace de sortie : vecteur compact de 512 nombres réels (l' <i>empreinte</i>)
$\ f(x)\ _2 = 1$	Normalisation : $\sqrt{f_1^2 + \dots + f_{512}^2} = 1$, tous les embeddings sont sur une sphère

Intuition — la sphère comme espace de comparaison

Imaginez que chaque visage soit représenté par un point sur la surface d'une boule géante en 512 dimensions. Deux photos de la même personne sont deux points très proches sur cette boule. Deux personnes différentes sont éloignées.



La normalisation $\|f(x)\|_2 = 1$ assure que la comparaison entre deux embeddings se ramène à mesurer *un angle* (produit scalaire), ce qui est numériquement stable et indépendant de la luminosité de l'image.

Lien avec le code Python

Listing 4: Normalisation L2 de l'embedding (P3)

```

1 import numpy as np
2
3 def l2_normalize(embedding):
4     """
5     Normalise un vecteur d'embedding pour que sa norme soit 1.
6     - embedding : tableau (512,) ou (N, 512)
7     """
8     norm = np.linalg.norm(embedding, axis=-1, keepdims=True)
9     # norm = sqrt(e[0]^2 + e[1]^2 + ... + e[511]^2)

```

```

10
11     return embedding / norm      # chaque element divise par la
        norme
12
13 def cosine_similarity(emb1, emb2):
14     """
15     Similarite cosinus entre deux embeddings normalises.
16     Si les deux sont normalises (norme=1), le produit scalaire
17     est directement la similarite cosinus.
18     """
19     # Produit scalaire : sum(e1[i] * e2[i]) pour i in 0..511
20     return np.dot(emb1, emb2)    # entre -1 et 1
21
22 # Exemple :
23 emb_patrick1 = np.random.randn(512)    # sortie brute du reseau
24 emb_patrick2 = np.random.randn(512)
25 emb_lisbon   = np.random.randn(512)
26
27 # Normalisation
28 e1 = l2_normalize(emb_patrick1)
29 e2 = l2_normalize(emb_patrick2)
30 e3 = l2_normalize(emb_lisbon)
31
32 print(f"Sim(Patrick1, Patrick2) = {cosine_similarity(e1,e2):.3f}
        ")
33 # On s'attend a une valeur elevee apres entraInement
34 print(f"Sim(Patrick,  Lisbon)   = {cosine_similarity(e1,e3):.3f}
        ")
35 # On s'attend a une valeur faible

```

3.2 Triplet Loss

$$\mathcal{L} = \sum_{i=1}^N \max\left(\underbrace{\|f(x_i^a) - f(x_i^p)\|_2^2}_{\text{distance ancre-positif}} - \underbrace{\|f(x_i^a) - f(x_i^n)\|_2^2}_{\text{distance ancre-négatif}} + \alpha, 0\right) \quad (4)$$

Symbole	Signification
x_i^a	Ancre : photo de référence (ex. Patrick, face caméra)
x_i^p	Positif : autre photo de <i>la même</i> personne (Patrick, profil)
x_i^n	Négatif : photo d'une <i>autre</i> personne (Lisbon)
$\ f(x^a) - f(x^p)\ _2^2$	Distance euclidienne au carré entre les embeddings : à minimiser
$\ f(x^a) - f(x^n)\ _2^2$	Distance euclidienne au carré : à maximiser
$\alpha > 0$	Marge de sécurité (souvent 0.2) : le négatif doit être plus loin que le positif <i>d'au moins</i> α
$\max(\cdot, 0)$	Si déjà satisfait, la perte vaut 0 (on n'update pas ces poids)

Intuition — un entraînement par comparaison de triplets

Le réseau reçoit 3 images à la fois et apprend une seule règle :

$$d(a, p) + \alpha < d(a, n)$$

« La photo du même Patrick doit être plus proche de l'ancre que la photo de Lisbon, avec une marge de sécurité α . »

$$\begin{array}{lll} x^a \text{ (ancre)} & \longleftrightarrow & x^p \text{ (Patrick)} \quad \text{distance } d_p \\ x^a \text{ (ancre)} & \longleftrightarrow & x^n \text{ (Lisbon)} \quad \text{distance } d_n \end{array}$$

But : $d_n - d_p > \alpha$. Si c'est déjà le cas : perte = 0, rien à faire. Sinon : on corrige les poids pour rapprocher p et éloigner n .

Lien avec le code Python

Listing 5: Triplet Loss en NumPy puis PyTorch (P3)

```

1 import numpy as np
2
3 # ---- Version NumPy (pour comprendre la formule) ----
```

```

4 def triplet_loss_numpy(f_a, f_p, f_n, alpha=0.2):
5     """
6     f_a, f_p, f_n : tableaux (N, 512) -- N triplets du batch
7     alpha : marge de securite
8     """
9     # Distance euclidienne au carre entre ancre et positif
10    d_pos = ((f_a - f_p) ** 2).sum(axis=1) # forme (N,)
11
12    # Distance euclidienne au carre entre ancre et negatif
13    d_neg = ((f_a - f_n) ** 2).sum(axis=1) # forme (N,)
14
15    # Eq.(3) : max(d_pos - d_neg + alpha, 0)
16    loss_per_triplet = np.maximum(d_pos - d_neg + alpha, 0)
17
18    return loss_per_triplet.mean() # moyenne sur le batch
19
20 # Exemple numerique simple (vecteurs 3D pour la lisibilite) :
21 a = np.array([[1.0, 0.0, 0.0]]) # ancre
22 p = np.array([[0.9, 0.1, 0.0]]) # positif (proche)
23 n = np.array([[0.0, 1.0, 0.0]]) # negatif (loin)
24
25 d_pos = ((a-p)**2).sum() # = 0.01 + 0.01 = 0.02
26 d_neg = ((a-n)**2).sum() # = 1.0 + 1.0 = 2.0
27 print(f"d_pos={d_pos:.2f}, d_neg={d_neg:.2f}")
28 print(f"Loss = max({d_pos:.2f} - {d_neg:.2f} + 0.2, 0) = {max(
29     d_pos-d_neg+0.2, 0):.2f}")
30
31 # Loss = max(0.02 - 2.0 + 0.2, 0) = max(-1.78, 0) = 0.0 ->
32     triplet deja satisfait!
33
34 # ---- Version PyTorch (utilisee dans le projet) ----
35 # import torch, torch.nn as nn
36 # criterion = nn.TripletMarginLoss(margin=0.2, p=2)
37 # loss = criterion(f_anchor, f_positive, f_negative)

```

4 Classification SVM Multi-Classes

4.1 Problème d'optimisation Soft Margin

$$\min_{w, b, \xi} \underbrace{\frac{1}{2} \|w\|^2}_{\text{maximise la marge}} + C \underbrace{\sum_{i=1}^n \xi_i}_{\text{pénalise les erreurs}} \quad (5)$$

Symbole	Signification
w	Vecteur normal à l'hyperplan séparateur (définit l'orientation de la frontière)
b	Biais : déplace l'hyperplan sans changer son orientation
$\xi_i \geq 0$	Variable d'écart de l'exemple i : $\xi_i = 0$ (bon), $0 < \xi_i \leq 1$ (dans la marge), $\xi_i > 1$ (mal classé)
$\frac{1}{2} \ w\ ^2$	Régularisation : minimiser $\ w\ \equiv$ maximiser la marge $\frac{2}{\ w\ }$
$C > 0$	Compromis tolérance/marge : C grand $\rightarrow$ peu d'erreurs tolérées, C petit $\rightarrow$ marge large

Intuition — la marge comme couloir de sécurité

Imaginez deux groupes de points sur une feuille. Le SVM cherche la droite (en 2D) ou l'hyperplan (en haute dimension) qui les sépare avec le *couloir le plus large possible* de chaque côté.

```

o o o |   | x x
o o   |   | x x
      <----->
      marge = 2 / ||w||

```

Les points dans le couloir violent la marge : leur $\xi_i > 0$. Le paramètre C décide si on préfère un couloir large (tolérant quelques violations) ou un couloir étroit sans erreur. Pour la reconnaissance, C est un hyper-paramètre à choisir par validation croisée.

Lien avec le code Python

Listing 6: SVM multi-classes avec sklearn (P4)

```
1 from sklearn.svm import SVC
```

```

2 from sklearn.preprocessing import normalize
3 import numpy as np
4
5 # --- Donnees simulees (remplacer par les vrais embeddings de
    FaceNet) ---
6 # X_train : (N, 512) -- embeddings FaceNet normalises
7 # y_train : (N,)      -- labels 0=Patrick, 1=Lisbon, 2=Cho, 3=
    autre
8
9 # Etape 1 : s'assurer que les embeddings sont normalises (norme
    L2 = 1)
10 # (FaceNet le fait deja, mais verification utile)
11 X_train_norm = normalize(X_train, norm='l2') # chaque ligne
    divisee par sa norme
12
13 # Etape 2 : creer le SVM multi-classes
14 # - decision_function_shape='ovo' : One-vs-One (strategy decrite
    eq.(4))
15 # - C=1.0 : compromis standard, a tuner
16 # - kernel='rbf' : noyau Gaussien pour separation non lineaire
17 clf = SVC(
18     C=1.0,
19     kernel='rbf',
20     decision_function_shape='ovo', # One-vs-One :  $K=N*(N-1)/2$ 
        classifieurs
21     probability=True              # pour avoir des scores de
        confiance
22 )
23
24 # Etape 3 : entraînement
25 clf.fit(X_train_norm, y_train)
26 # Sklearn entraine automatiquement  $K=4*3/2=6$  classifieurs
    binaires
27
28 # Etape 4 : prediction (= vote majoritaire interne)
29 X_test_norm = normalize(X_test, norm='l2')
30 y_pred = clf.predict(X_test_norm)
31 # y_pred[i] = classe avec le + de votes parmi les 6 duels (eq
    .(4))
32
33 # Etape 5 : inspecter le C optimal par validation croisee

```



```

34 from sklearn.model_selection import GridSearchCV
35 param_grid = {'C': [0.01, 0.1, 1, 10, 100]}
36 grid = GridSearchCV(SVC(kernel='rbf', decision_function_shape='
    ovo'),
37                     param_grid, cv=5, scoring='accuracy')
38 grid.fit(X_train_norm, y_train)
39 print(f"Meilleur C = {grid.best_params_['C']}")

```

4.2 Stratégie One-vs-One (OvO)

$$\hat{y} = \arg \max_{c \in \{1 \dots N\}} \left(\sum_{j < k} \mathbb{1}\{f_{j,k}(x) = c\} \right) \quad (6)$$

Symbole	Signification
$K = \frac{N(N-1)}{2}$	Nombre de duels binaires : pour $N = 4$, $K = 6$
$f_{j,k}(x)$	Classifieur binaire du duel j contre k : retourne j ou k
$\mathbb{1}\{f_{j,k}(x) = c\}$	Vaut 1 si le duel donne c comme gagnant, 0 sinon (un vote)
$\arg \max_c$	Classe qui a reçu le plus de votes au total

Exemple complet avec $N = 4$ identités. Le visage à reconnaître appartient à Patrick (classe 1). Les 6 classifieurs donnent :

Duel	Gagnant	Vote pour Patrick
$f_{1,2}$: Patrick vs Lisbon	Patrick	+1
$f_{1,3}$: Patrick vs Cho	Patrick	+1
$f_{1,4}$: Patrick vs classe 4	Patrick	+1
$f_{2,3}$: Lisbon vs Cho	Lisbon	0
$f_{2,4}$: Lisbon vs classe 4	Lisbon	0
$f_{3,4}$: Cho vs classe 4	Cho	0

Scores : Patrick = 3, Lisbon = 2, Cho = 1, classe 4 = 0. $\hat{y} = \arg \max = \mathbf{Patrick}$.
✓

5 Chiffres Manuscrits : Extraction de Features (P5)

5.1 De l'image au vecteur

Une image MNIST est une grille 28×28 de niveaux de gris $\in [0, 255]$, normalisés en $[0, 1]$. Aplatie, elle donne un vecteur $\mathbf{x} \in [0, 1]^{784}$.

Intuition — 784 dimensions c'est trop

784 features pour 1000 exemples d'entraînement, c'est le problème du « fléau de la dimensionnalité » : le SVM devient lent et les distances perdent leur sens. La PCA permet de compresser en ne gardant que les directions qui *expliquent le plus de variance*.

5.2 Réduction de dimension par PCA

La PCA cherche les directions $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k$ (appelées **composantes principales**) qui capturent le maximum de variance des données.

$$\mathbf{z} = \mathbf{V}_k^\top (\mathbf{x} - \boldsymbol{\mu}) \quad (7)$$

Symbole	Signification
$\mathbf{x} \in \mathbb{R}^{784}$	Vecteur pixel brut
$\boldsymbol{\mu} \in \mathbb{R}^{784}$	Moyenne des données d'entraînement (pixel moyen)
$\mathbf{x} - \boldsymbol{\mu}$	Centrage : on soustrait la moyenne pour avoir des données centrées
$\mathbf{V}_k \in \mathbb{R}^{784 \times k}$	Matrice des k premiers vecteurs propres (composantes principales)
$\mathbf{z} \in \mathbb{R}^k$	Représentation compressée ($k \ll 784$, ex. $k = 50$)

Intuition — la PCA comme ombre sur un mur

Imaginez des points en 3D qui forment un nuage aplati comme une crêpe. La PCA trouve l'orientation de la crêpe (le plan qui contient le plus d'information) et

projette tous les points sur ce plan. On passe de 3D à 2D sans presque rien perdre.

Pour MNIST, on passe de 784D à 50D et on conserve typiquement $\sim 90\%$ de la variance totale.

Lien avec le code Python

Listing 7: PCA sur MNIST puis SVM (P5 + P4)

```
1 from sklearn.decomposition import PCA
2 from sklearn.svm import SVC
3 from sklearn.datasets import fetch_openml
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import normalize
6 import numpy as np
7
8 # --- Chargement MNIST ---
9 mnist = fetch_openml('mnist_784', version=1, as_frame=False)
10 X, y = mnist.data / 255.0, mnist.target.astype(int)
11 X_train, X_test, y_train, y_test = train_test_split(
12     X, y, test_size=0.2, random_state=42)
13
14 # --- PCA : reduction 784D -> 50D (P5) ---
15 pca = PCA(n_components=50)      # on garde les 50 premières
    composantes
16
17 # fit : calcule mu et les vecteurs propres V_k sur les donnees d
    'entraînement'
18 pca.fit(X_train)
19
20 # transform :  $z = V_k^T (x - \mu)$  -- eq.(5)
21 X_train_pca = pca.transform(X_train)    # forme (N_train, 50)
22 X_test_pca  = pca.transform(X_test)     # meme transformation
23
24 # Variance expliquée : combien d'info on a garde
25 print(f"Variance expliquée : {pca.explained_variance_ratio_.sum
    ()*100:.1f}%")
26
27 # --- SVM sur les features compressees (P4) ---
28 clf = SVC(C=10, kernel='rbf', decision_function_shape='ovo')
29 clf.fit(X_train_pca, y_train)
30 acc = clf.score(X_test_pca, y_test)
31 print(f"Accuracy : {acc*100:.2f}%")      # attendu : ~97-98%
```

6 Évaluation & Métriques (P6)

6.1 Matrice de confusion et métriques dérivées

$$\text{Précision} = \frac{VP}{VP + FP}, \quad \text{Rappel} = \frac{VP}{VP + FN}, \quad F_1 = 2 \cdot \frac{\text{Prec} \times \text{Rappel}}{\text{Prec} + \text{Rappel}} \quad (8)$$

Symbole	Signification
VP	Vrais Positifs : prédit Patrick, c'est bien Patrick
FP	Faux Positifs : prédit Patrick, c'est en réalité quelqu'un d'autre
FN	Faux Négatifs : c'est Patrick mais le système ne l'a pas reconnu
Précision	Sur tous les cas où on dit « Patrick », quelle fraction est correcte ?
Rappel	Sur tous les vrais Patrick, quelle fraction a été retrouvée ?
F_1	Moyenne harmonique Précision/Rappel (pénalise si l'une est faible)

Intuition — précision vs rappel : le détecteur de spam

Précision haute : quand je dis « c'est du spam », je me trompe rarement. Mais je laisse peut-être passer beaucoup de vrais spams.

Rappel haut : je détecte presque tous les spams. Mais je signale parfois à tort des mails légitimes.

Le F_1 équilibre les deux. Pour la reconnaissance faciale, un mauvais rappel = des personnes non reconnues. Une mauvaise précision = de fausses identifications (plus grave en sécurité).

Lien avec le code Python

Listing 8: Calcul complet des métriques (P6)

```
1 from sklearn.metrics import (classification_report,
    confusion_matrix,
```

```

2         roc_auc_score, RocCurveDisplay)
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 # --- Matrice de confusion ---
7 cm = confusion_matrix(y_test, y_pred)
8 # cm[i,j] = nombre d'exemples de la classe i predits comme
   classe j
9 # La diagonale = bonnes predictions (VP de chaque classe)
10
11 print("Matrice de confusion :")
12 print(cm)
13
14 # --- Rapport complet ---
15 # Affiche precision, rappel, F1 pour chaque classe
16 print(classification_report(y_test, y_pred,
17     target_names=['Patrick', 'Lisbon', 'Cho', 'Autre']))
18
19 # --- AUC (pour la reconnaissance faciale, cas binaire) ---
20 # On transforme en probleme binaire : Patrick vs les autres
21 y_bin_true = (y_test == 0).astype(int) # 1 = Patrick, 0 =
   autre
22 y_scores = clf.predict_proba(X_test_norm)[: ,0] # score de
   confiance Patrick
23
24 auc = roc_auc_score(y_bin_true, y_scores)
25 print(f"AUC Patrick vs reste : {auc:.3f}")
26 # AUC=1.0 : parfait / AUC=0.5 : aleatoire
27
28 # --- Courbe ROC ---
29 RocCurveDisplay.from_predictions(y_bin_true, y_scores,
30     name="Patrick vs reste")
31 plt.title("Courbe ROC - Reconnaissance Patrick")
32 plt.savefig("roc_curve.pdf")
33
34 # --- Validation croisee (estimation robuste) ---
35 from sklearn.model_selection import cross_val_score
36 scores = cross_val_score(clf, X_train_pca, y_train, cv=5,
37     scoring='f1_macro')
37 print(f"F1 moyen (5-fold CV) : {scores.mean():.3f} +/- {scores.
   std():.3f}")

```

Références

1. K. Zhang et al., “Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks,” *IEEE Signal Processing Letters*, 2016.
2. F. Schroff et al., “FaceNet: A Unified Embedding for Face Recognition and Clustering,” *CVPR*, 2015.
3. V. Vapnik, *The Nature of Statistical Learning Theory*. Springer, 1995.
4. Y. LeCun et al., “Gradient-based learning applied to document recognition,” *Proc. IEEE*, 1998. (MNIST)
5. C. Dalal & B. Triggs, “Histograms of Oriented Gradients for Human Detection,” *CVPR*, 2005. (HOG)